VetAndes

Examen Parcial II - Version B

ESTUDIANTE

Josias Pari Mamani

Ingenieria de Sistemas | Lenguaje de Programacion II

ENTREGABLE DE EVIDENCIAS DE EJECUCION

REPOSITORIOS GITHUB

Backend: github.com/Pari820/vetandes-backend
Frontend: github.com/Pari820/vetandes-frontend

VERIFICACIONES DOCUMENTADAS

- Angular: npm run build - compilacion exitosa.
- Spring Boot: 8 pruebas automatizadas, sin fallos, BUILD SUCCESS.
- API: login, consultas, filtros y lectura de atenciones - HTTP 200.
- API: errores validados - HTTP 400, 401, 404 y 409; seguridad HTTP 403.
- Atencion AT-000005: subtotal S/ 175, descuento S/ 17.50, total S/ 157.50.
- Reporte: 4 atenciones validas, ingreso S/ 465 agrupado por especie.

ALCANCE

Cada captura incluye lo que se probo, observo y demuestra.

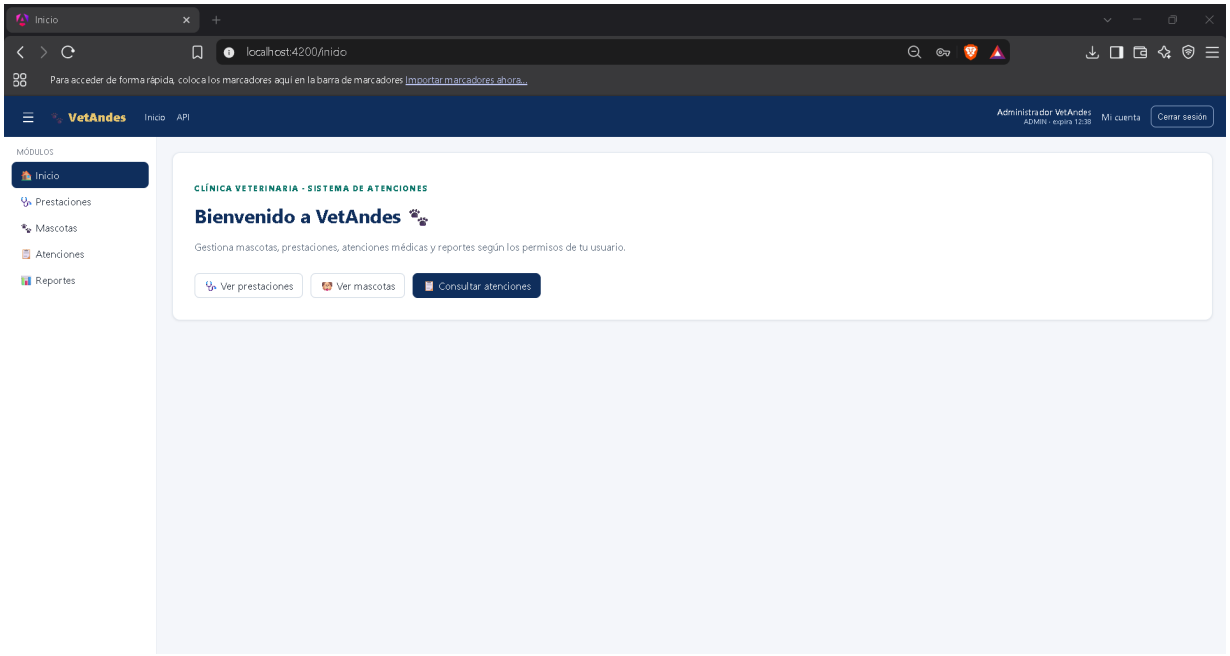
Prueba HTTP 201 y rollback transaccional: no documentados en las capturas.

29 capturas documentadas

30 de octubre de 2026

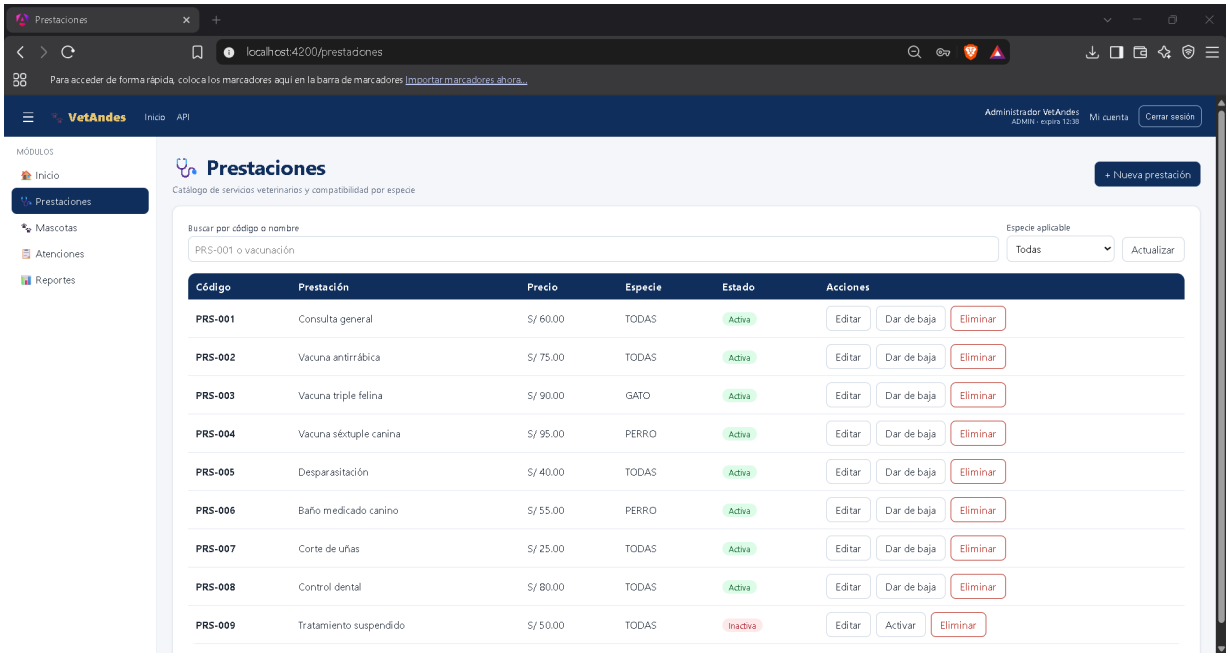
VetAndes | Examen Parcial II

01. Inicio de sesión como administrador



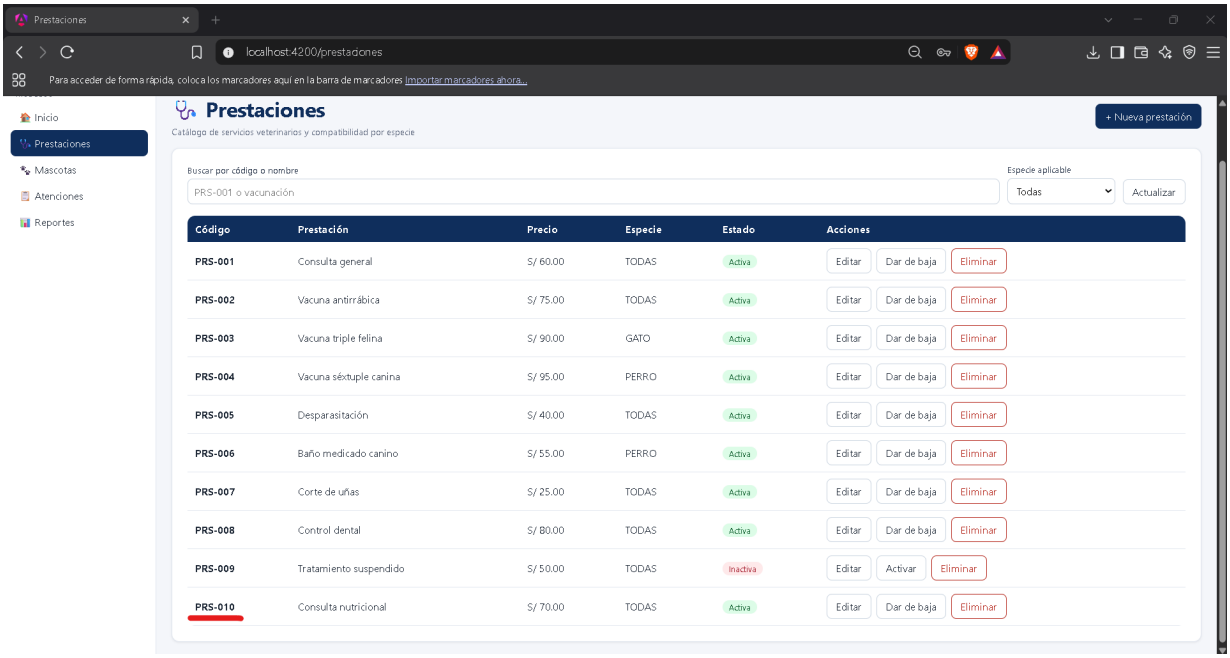
Se probó: Se inició sesión como admin en la SPA.
Se observó: Se mostró la pantalla de inicio, la identidad de administrador y los módulos disponibles.
Demuestra: El frontend presenta una sesión autenticada con el rol ADMIN.

02. Consulta de prestaciones



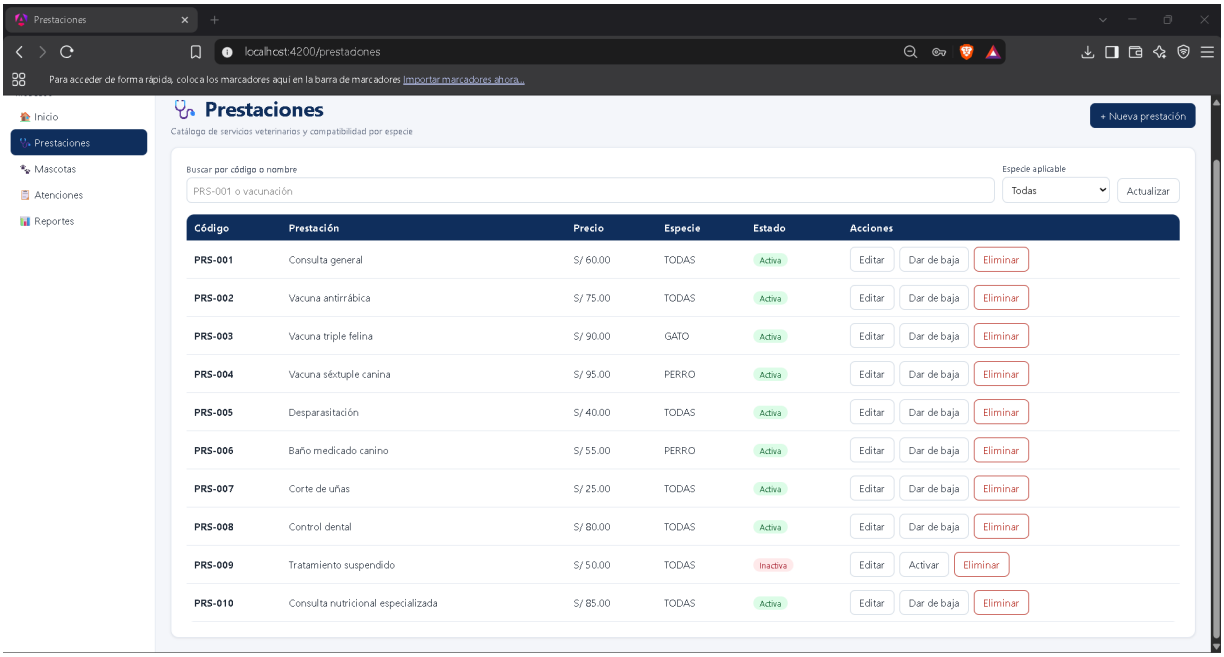
Se probó: Se abrió la lista de prestaciones del sistema.
Se observó: Se visualizaron nueve prestaciones semilla con código, precio, especie y estado.
Demuestra: Angular muestra el catálogo obtenido del backend, incluyendo una prestación inactiva.

03. Registro de prestación



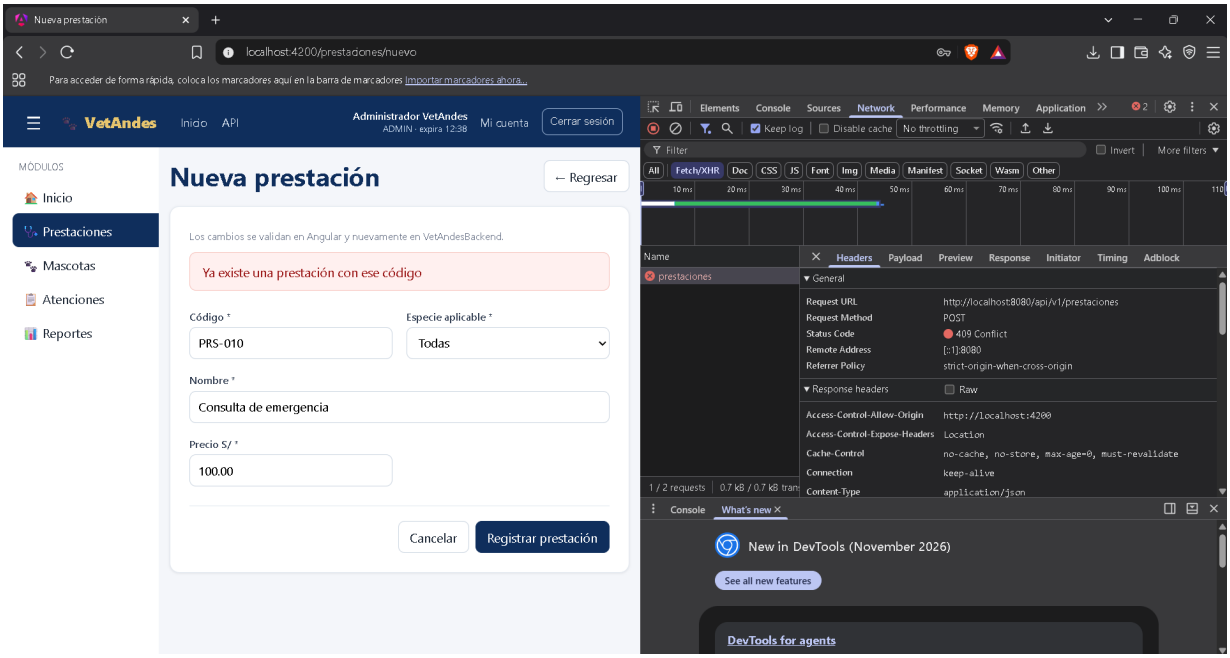
Se probó: Se registró PRS-010 como Consulta nutricional por S/ 70.00.
Se observó: Apareció la nueva prestación en la tabla como activa.
Demuestra: El alta de prestaciones se refleja en la lista del sistema.

04. Actualización de prestación



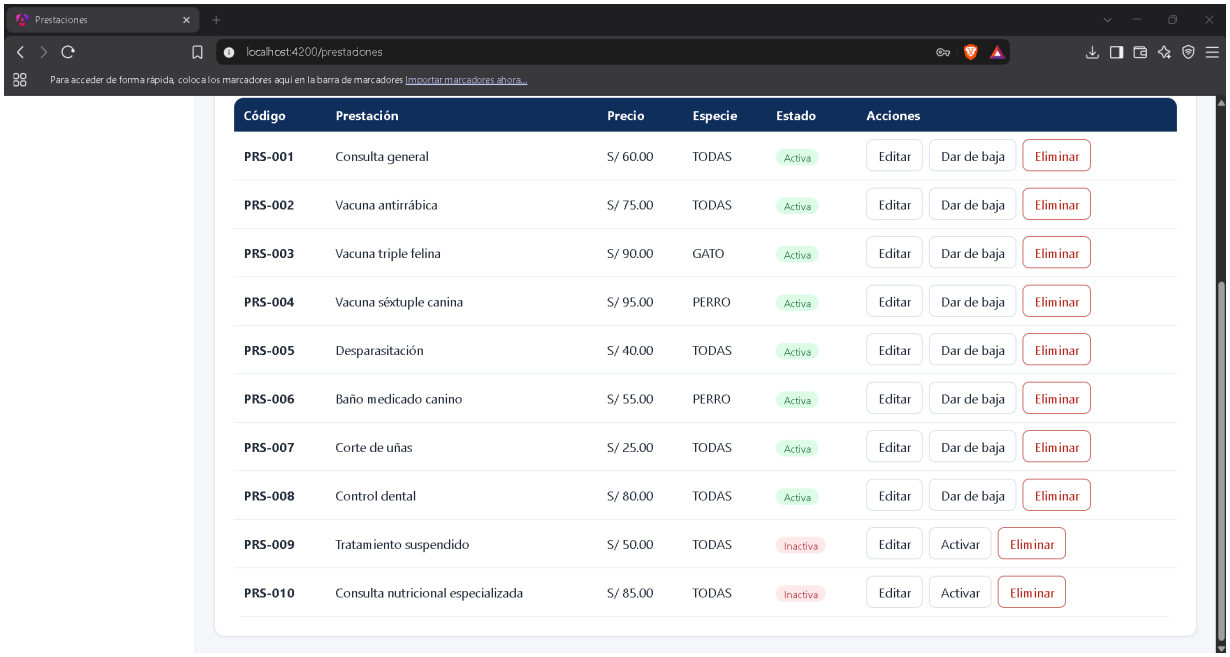
Se probó: Se modificaron el nombre y precio de PRS-010.
Se observó: La lista mostró Consulta nutricional especializada por S/ 85.00.
Demuestra: El formulario de edición actualiza los valores que muestra la SPA.

05. Rechazo por código duplicado (RN-01)



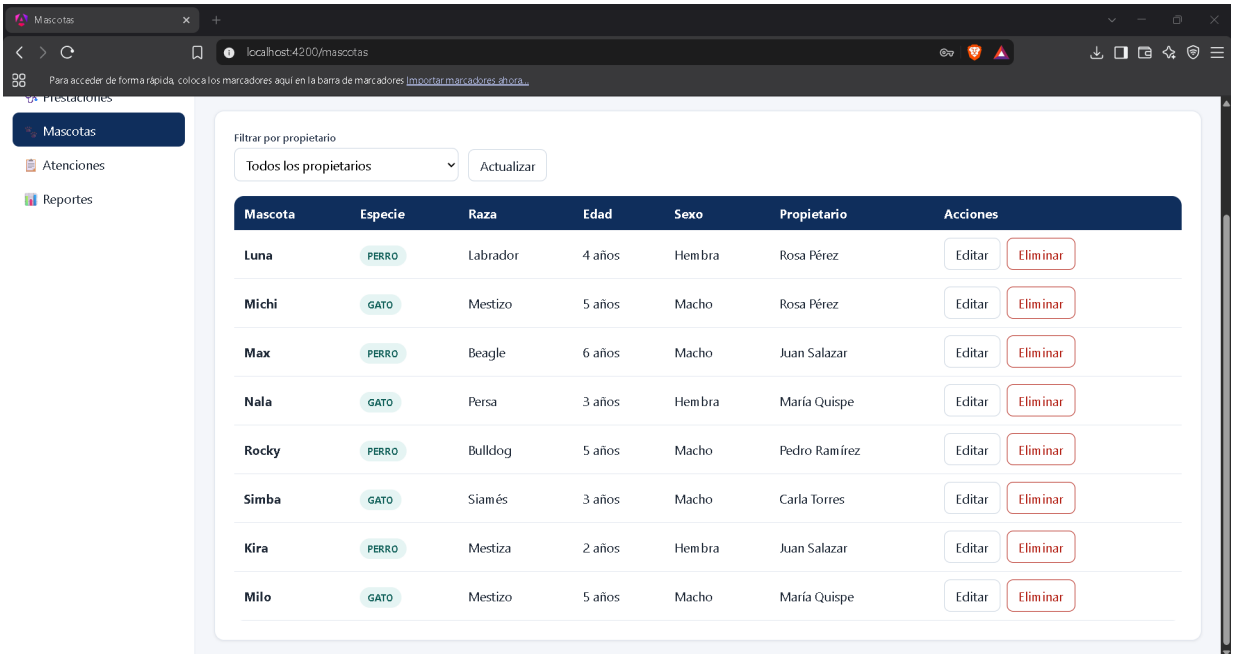
Se probó: Se intentó registrar otra prestación con el código PRS-010.
Se observó: Angular mostró el mensaje de duplicado y Network confirmó HTTP 409 Conflict.
Demuestra: El backend rechaza códigos duplicados y la SPA comunica el error al usuario.

06. Baja lógica de prestación



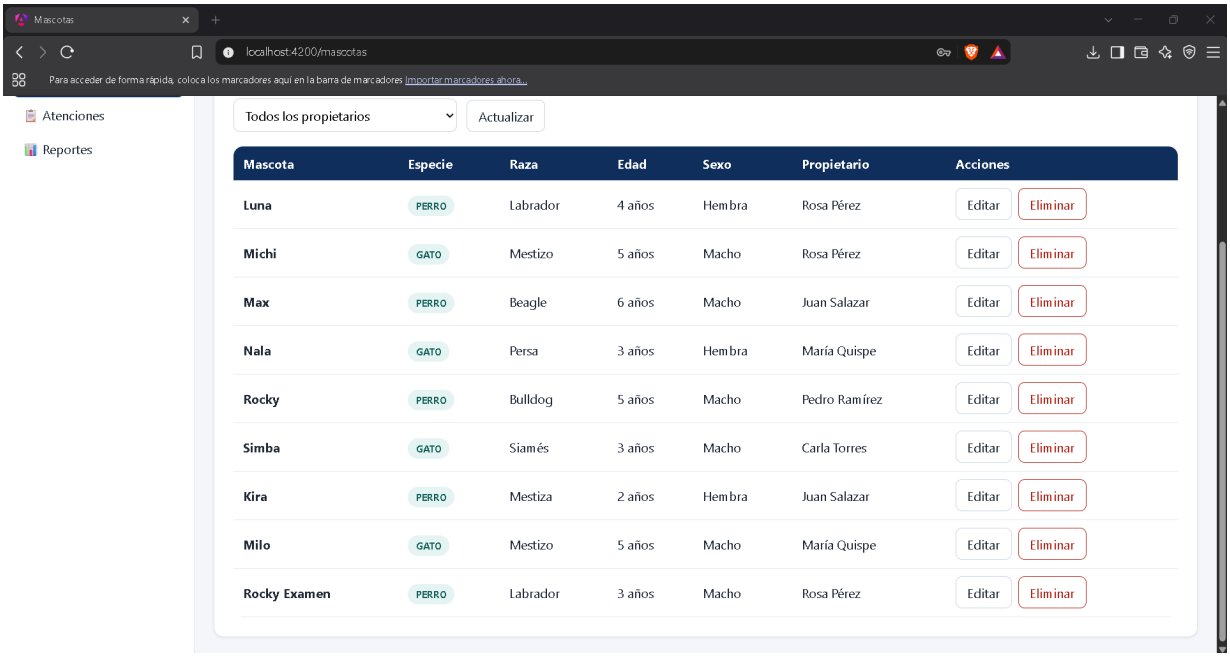
Se probó: Se dio de baja a PRS-010 desde el listado.
Se observó: PRS-010 continuó visible, ahora marcada como Inactiva.
Demuestra: El sistema actualiza el estado sin eliminar el registro de la lista.

07. Consulta de mascotas y propietarios



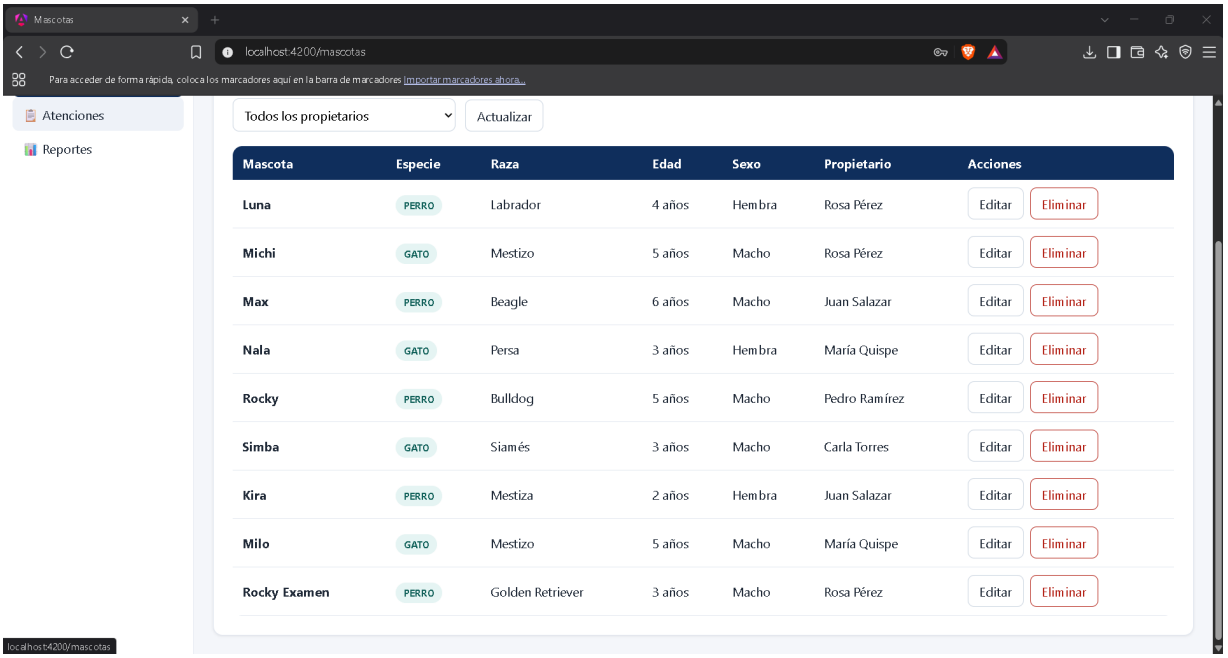
Se probó: Se consultó la lista de mascotas.
Se observó: Se visualizaron ocho mascotas con especie, raza, edad, sexo y propietario.
Demuestra: La SPA presenta datos asociados entre Mascota y Propietario.

08. Registro de mascota



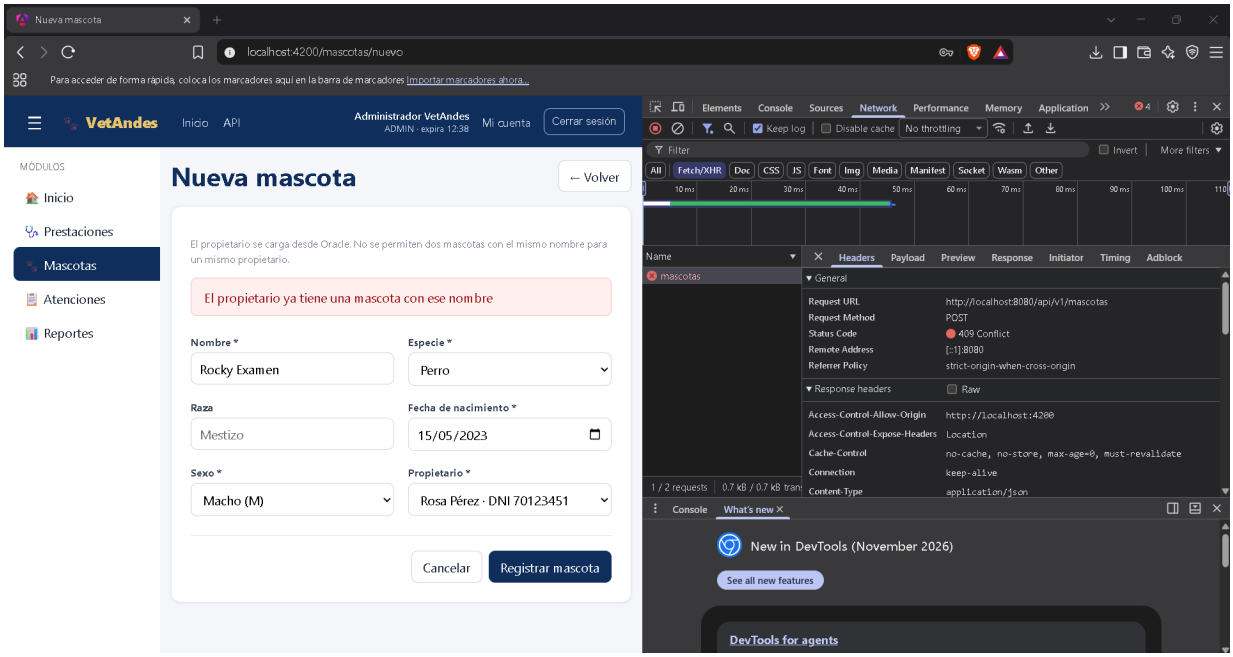
Se probó: Se registró Rocky Examen, perro Labrador, con Rosa Pérez como propietaria.
Se observó: La nueva mascota apareció junto a sus datos y el nombre de la propietaria.
Demuestra: El registro de mascotas muestra correctamente su asociación con un propietario.

09. Actualización de mascota



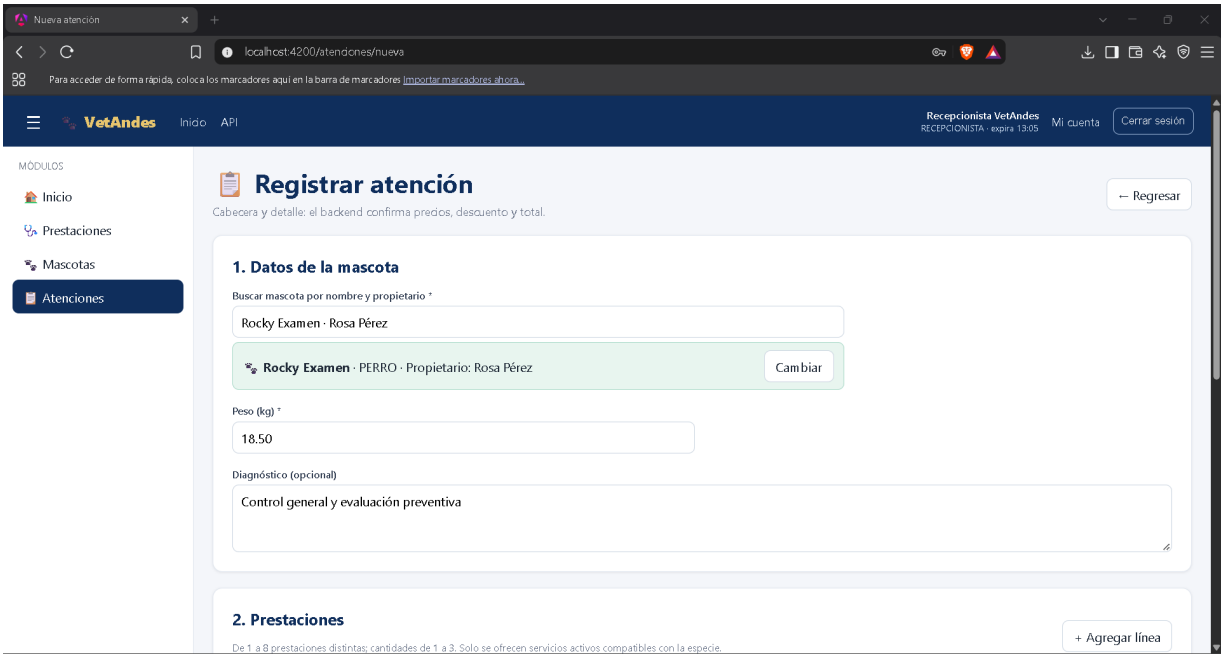
Se probó: Se cambió la raza de Rocky Examen a Golden Retriever.
Se observó: En la lista se mostró la raza actualizada y Rosa Pérez permaneció como propietaria.
Demuestra: El formulario de edición actualiza la mascota sin perder su relación con propietario.

10. Error 409 por mascota duplicada (RN-02)



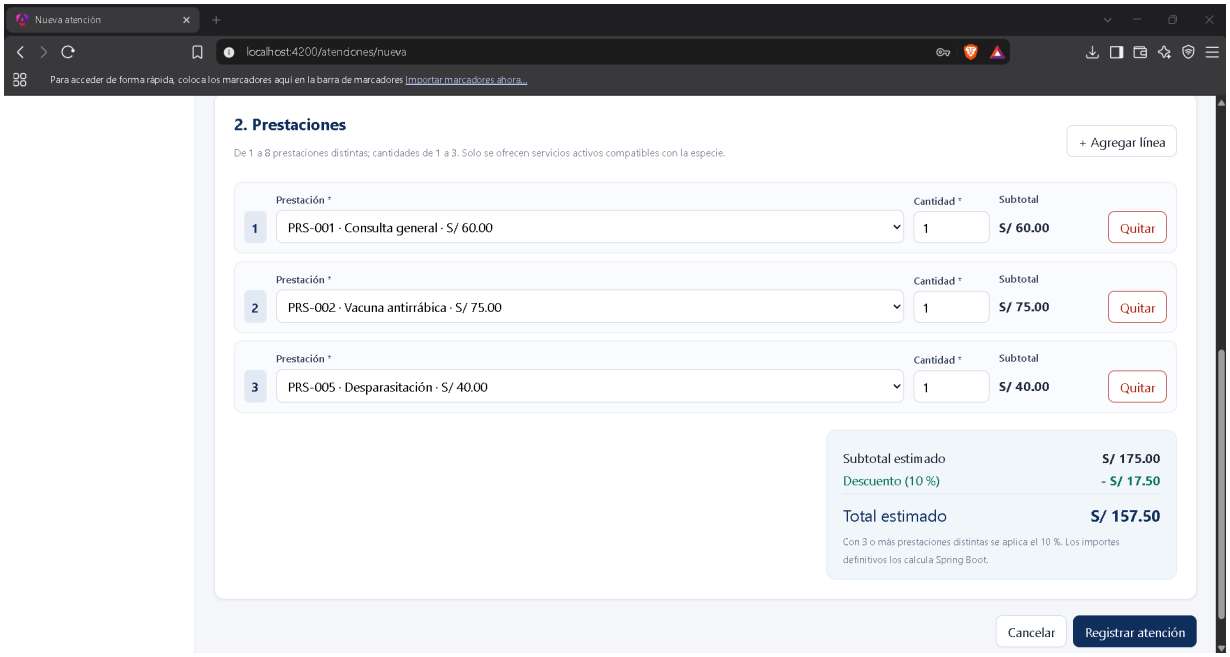
Se probó: Se intentó registrar otra mascota llamada Rocky Examen con la misma propietaria Rosa Pérez.
Se observó: El formulario presentó el mensaje de duplicidad y Network indicó HTTP 409 Conflict.
Demuestra: El backend exige unicidad del nombre por propietario y Angular muestra su rechazo.

11. Cabecera de atención desde Recepción



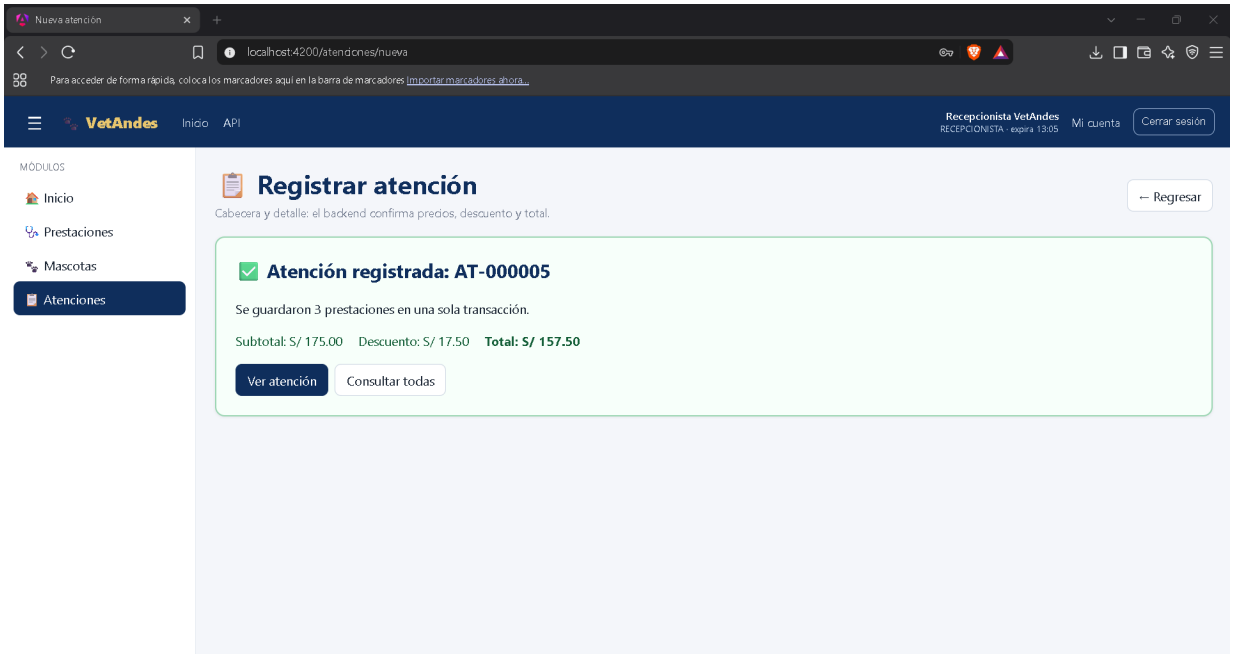
Se probó: Se ingresó como RECEPCIONISTA y se seleccionó a Rocky Examen para una nueva atención.
Se observó: Se muestra la mascota perro, la propietaria Rosa Pérez, el peso de 18.50 kg y el diagnóstico.
Demuestra: El formulario identifica la mascota y recopila los datos de cabecera para una atención.

12. Detalle de atención y descuento RN-04



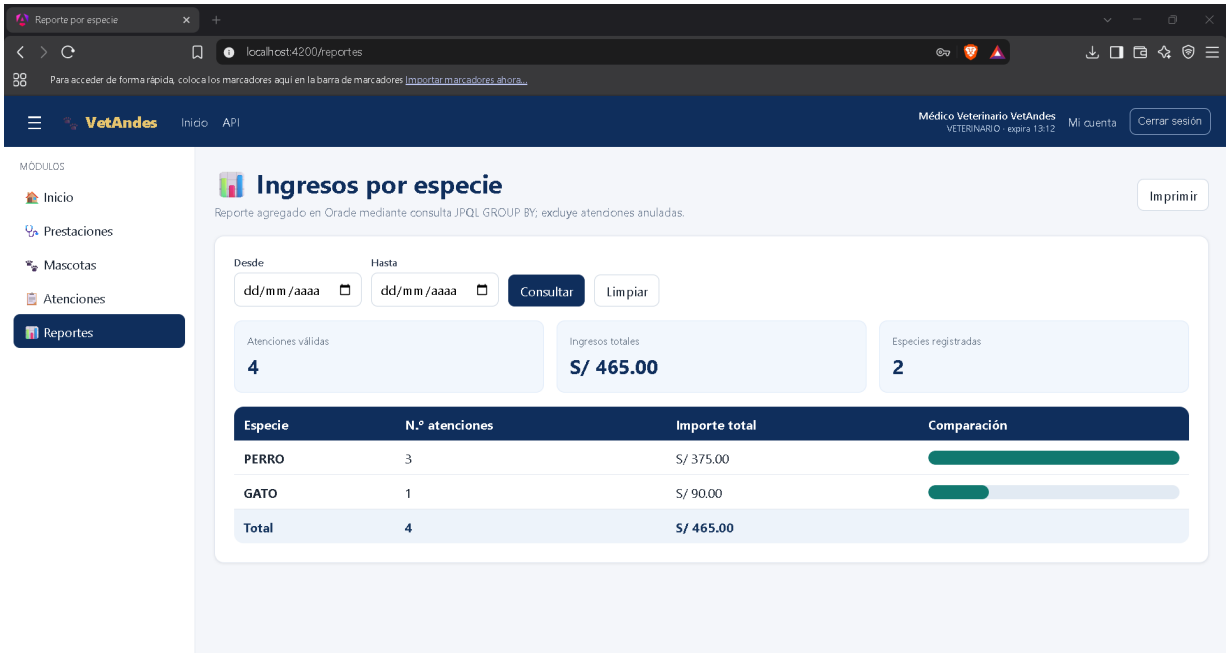
Se probó: Se seleccionaron tres prestaciones distintas: PRS-001, PRS-002 y PRS-005, una unidad de cada una.
Se observó: Angular muestra subtotal estimado S/ 175.00, descuento S/ 17.50 y total estimado S/ 157.50.
Demuestra: El formulario recalcula la vista previa del descuento por tres prestaciones. El importe definitivo debe validarse en el backend.

13. Atención registrada exitosamente (RN-04)



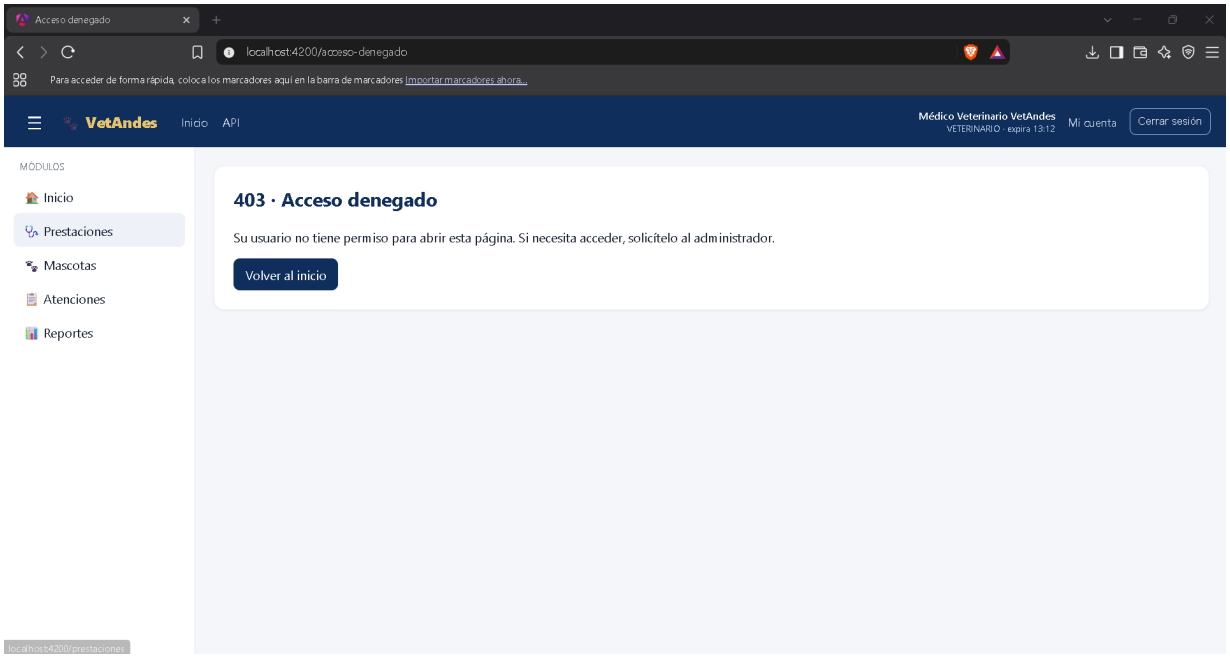
Se probó: Se registró como RECEPCIONISTA la atención de Rocky Examen con tres prestaciones.
Se observó: El backend devolvió el número AT-000005 y confirmó subtotal S/ 175.00, descuento S/ 17.50 y total S/ 157.50.
Demuestra: La atención se registró y el importe definitivo confirmado coincide con el cálculo esperado. La atomicidad se comprobará con una prueba de rollback.

14. Reporte de ingresos por especie



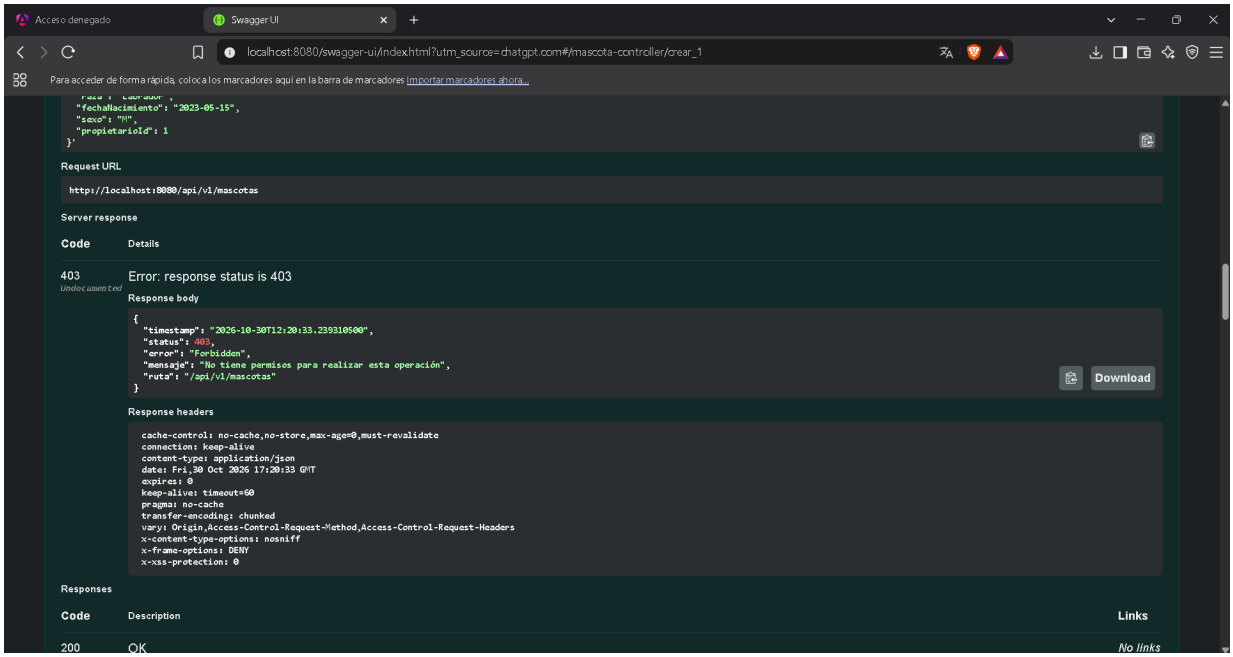
Se probó: Se consultó el reporte desde la sesión del usuario VETERINARIO.
Se observó: La pantalla mostró 3 atenciones de PERRO por S/ 375.00 y 1 de GATO por S/ 90.00; total S/ 465.00 en 4 atenciones válidas.
Demuestra: La SPA presenta resultados agrupados por especie y el usuario VETERINARIO tiene acceso al reporte. La consulta JPQL GROUP BY y exclusión de anuladas se revisan en código y pruebas.

15. Ruta restringida - Acceso denegado



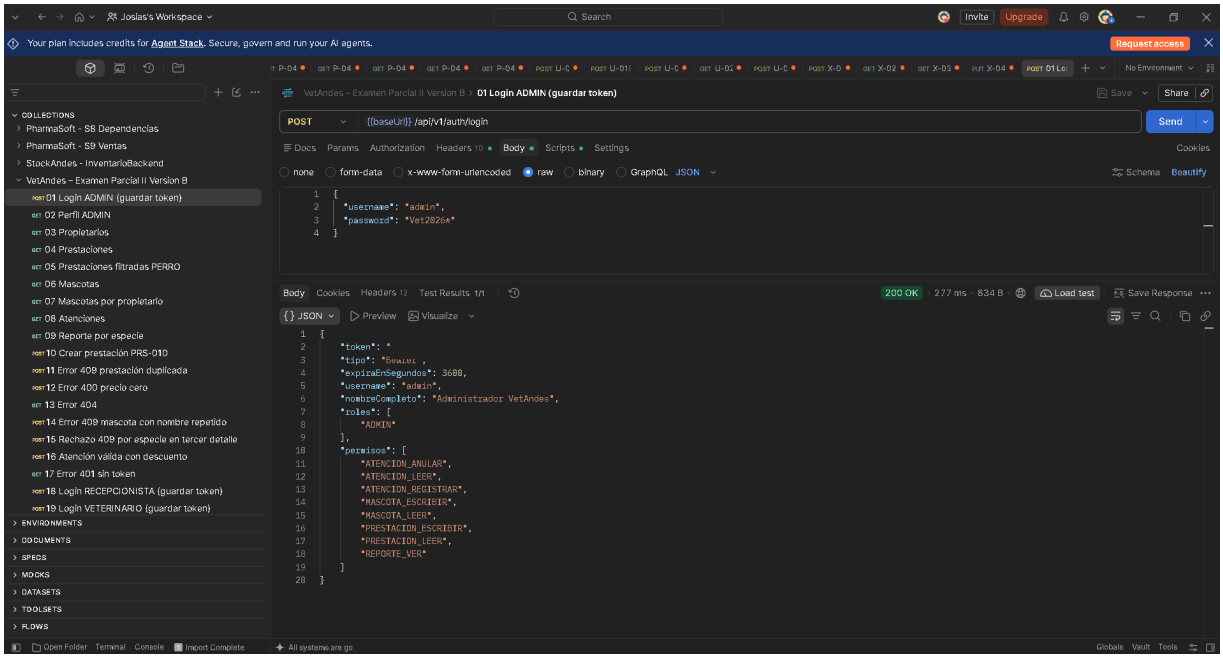
Se probó: Con la sesión de VETERINARIO activa, se intentó abrir manualmente /atenciones/nueva.
Se observó: Angular redirigió a /acceso-denegado, mostró 403 Acceso denegado y mantuvo activa la sesión del veterinario.
Demuestra: El permisoGuard impide navegar a una ruta no autorizada y el usuario conserva la sesión.

16. Backend - rechazo HTTP 403 en Swagger



Se probó: Con un token de VETERINARIO se intento crear una mascota mediante POST /api/v1/mascotas desde Swagger.
Se observó: La API devolvio HTTP 403 Forbidden, mensaje No tiene permisos para realizar esta operacion, ruta /api/v1/mascotas y formato de error comun.
Demuestra: Spring Security verifica los permisos en el backend, independientemente de los botones o rutas ocultos en Angular.

17. Login ADMIN en Postman - HTTP 200



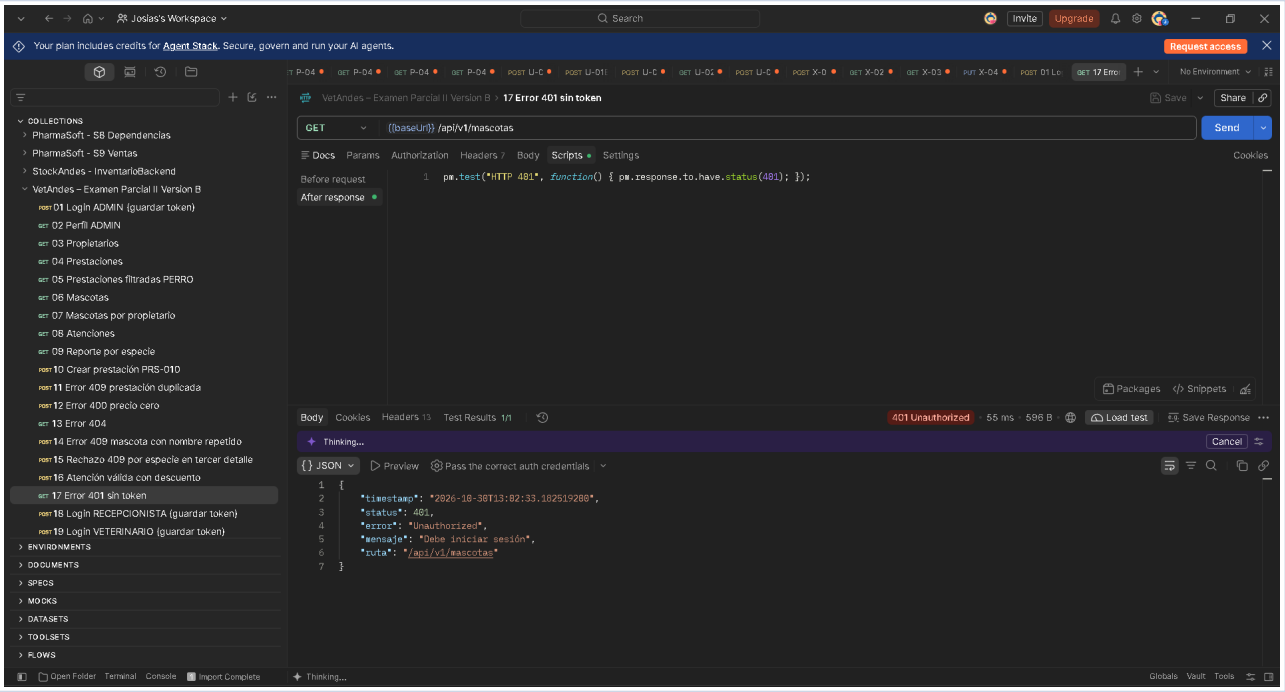
Se probó: Se ejecutó POST /api/v1/auth/login con las credenciales de prueba ADMIN en la colección importada.
Se observó: Postman devolvió HTTP 200 OK, identidad admin, rol ADMIN, permisos y resultado 1/1 de sus pruebas. Se ocultó el JWT de la captura.
Demuestra: La autenticación REST devuelve token y autorizaciones del usuario; el resultado HTTP 200 está confirmado.

18. Postman - HTTP 401 sin token

Se probó: GET /api/v1/mascotas sin token de autenticación.

Se observó: HTTP 401 Unauthorized, mensaje «Debe iniciar sesión» y prueba 1/1.

Demuestra: La API exige autenticación para consultar recursos protegidos.

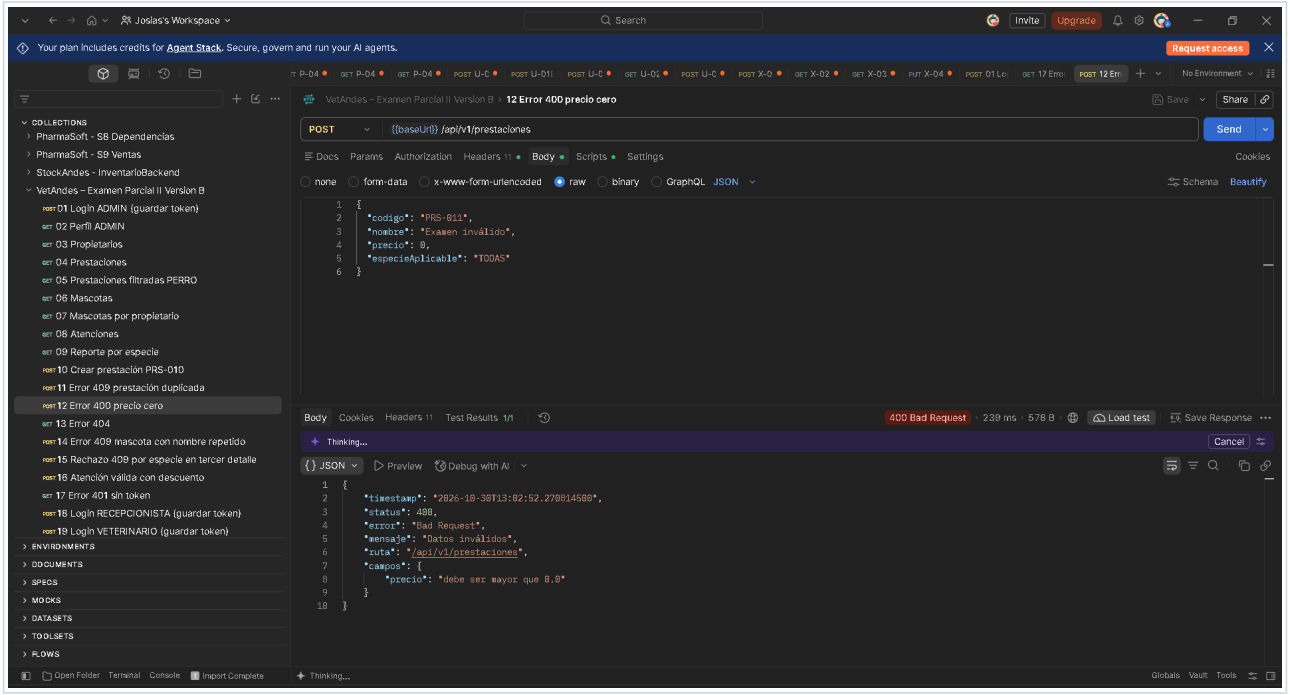


19. Postman - HTTP 400 precio cero

Se probó: POST /api/v1/prestaciones con precio igual a cero.

Se observó: HTTP 400 Bad Request y validación de precio mayor que 0.

Demuestra: Validación de datos y respuesta de error estructurada.

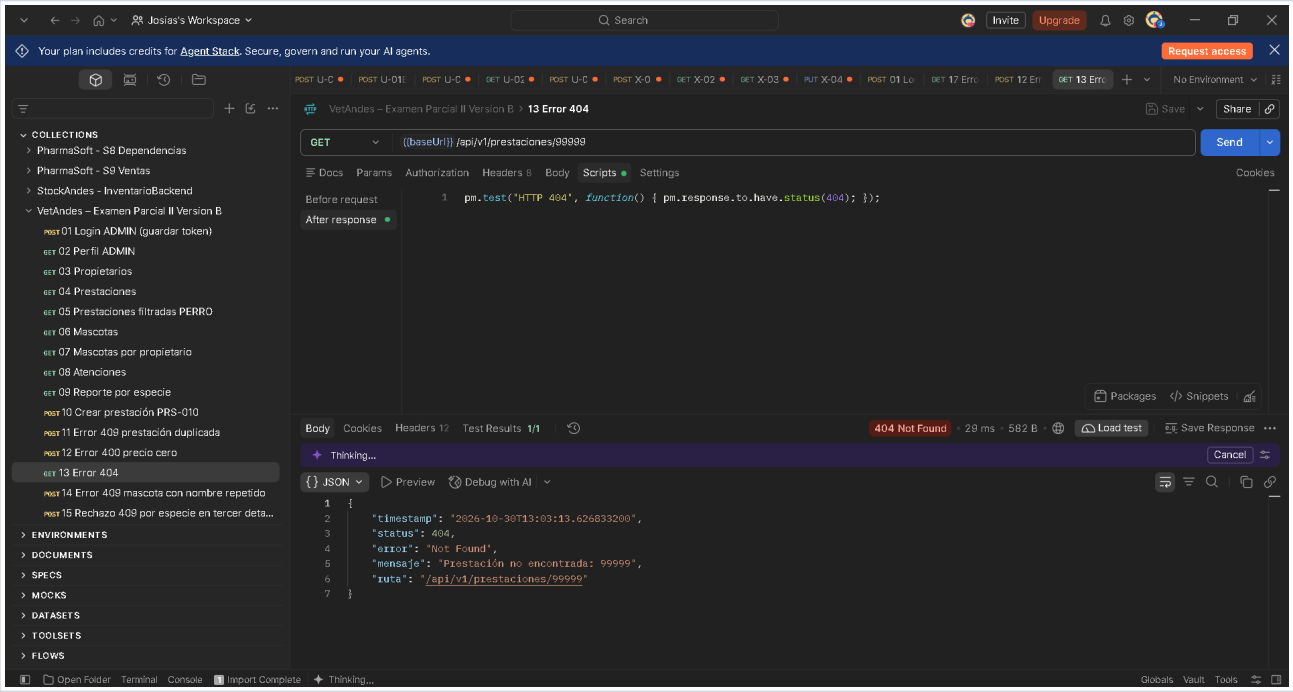


20. Postman - HTTP 404 recurso inexistente

Se probó: GET /api/v1/prestaciones/99999.

Se observó: HTTP 404 Not Found: prestación no encontrada.

Demuestra: El backend controla correctamente la búsqueda de recursos inexistentes.

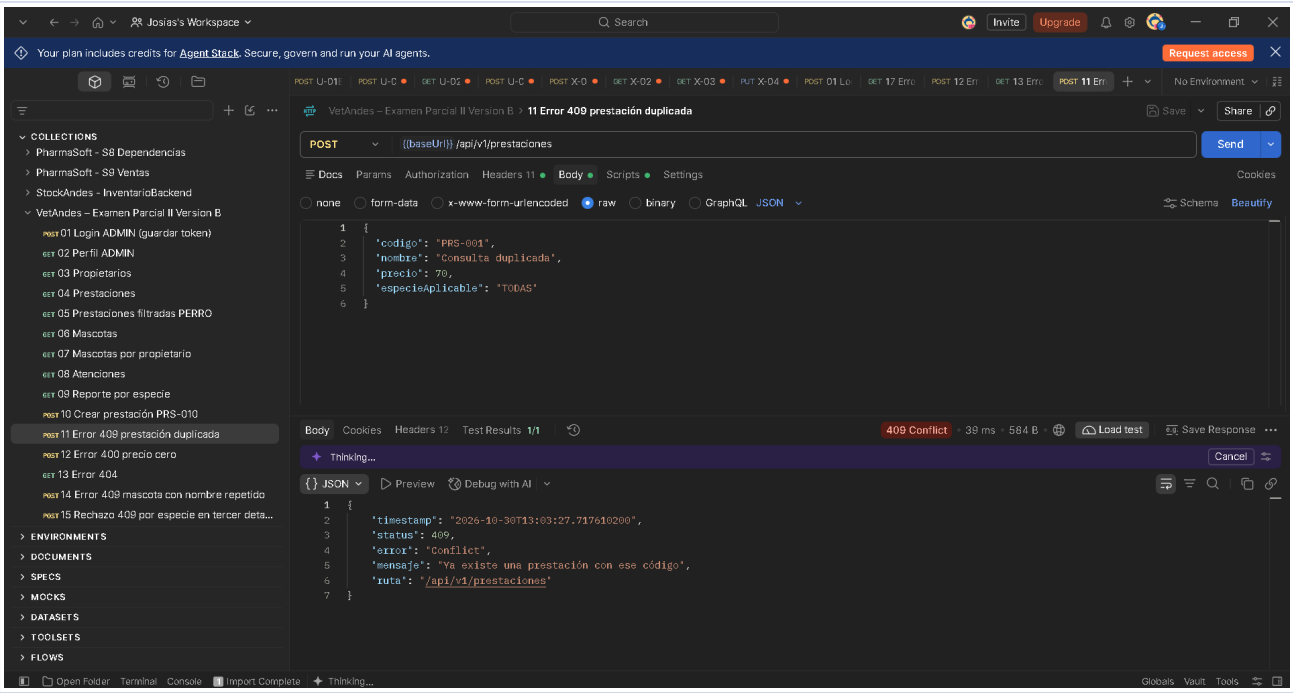


21. Postman - HTTP 409 código duplicado

Se probó: POST /api/v1/prestaciones con código PRS-001 ya existente.

Se observó: HTTP 409 Conflict y aviso de prestación con código duplicado.

Demuestra: La lógica del negocio impide registrar prestaciones duplicadas.

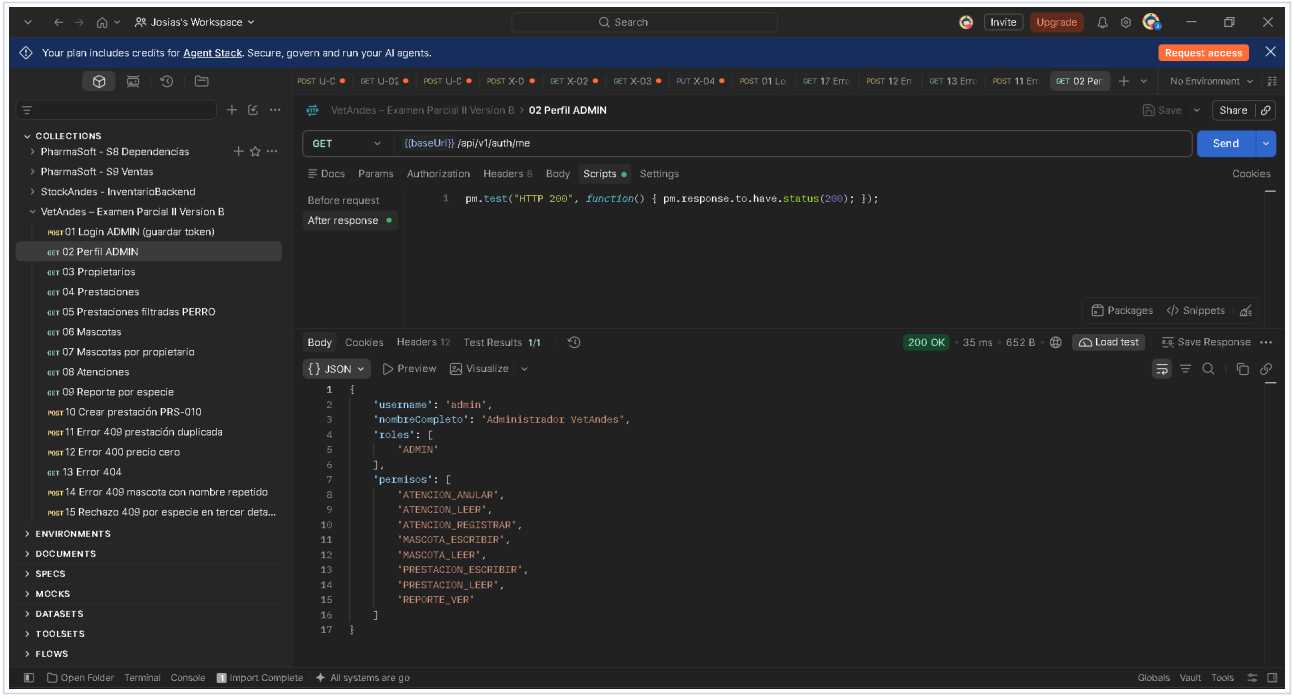


22. Postman - HTTP 200 perfil ADMIN

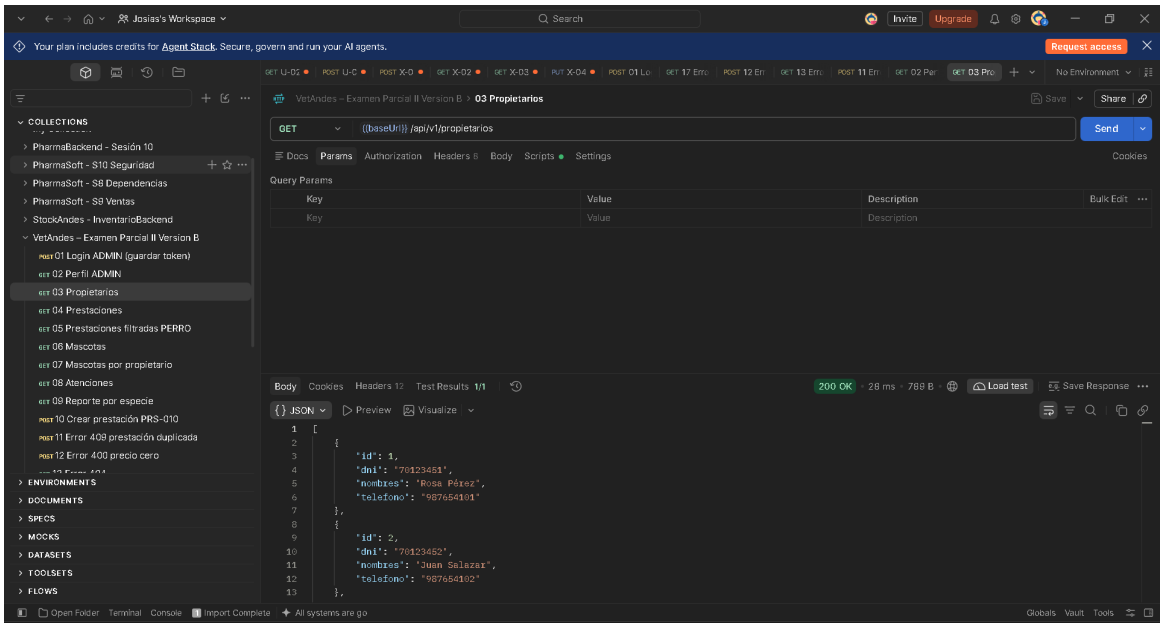
Se probó: GET /api/v1/auth/me con autenticación de administrador.

Se observó: HTTP 200 OK; usuario, rol ADMIN y permisos visibles.

Demuestra: JWT autorizado y consulta del perfil/permisos desde la API.

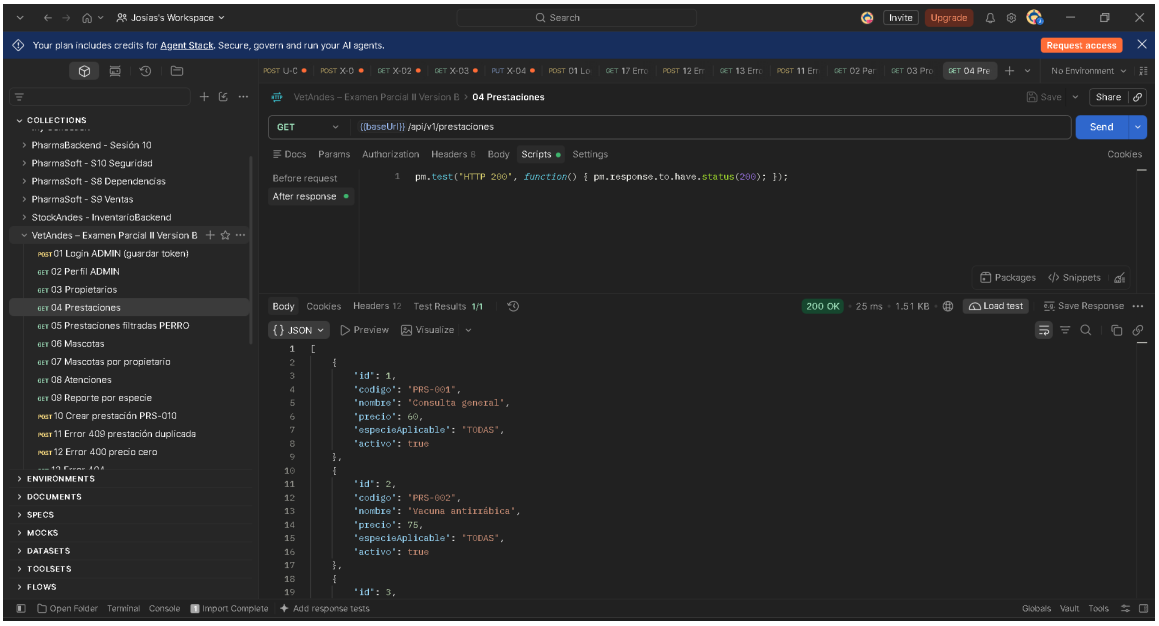


23. Postman - Consulta de propietarios, HTTP 200



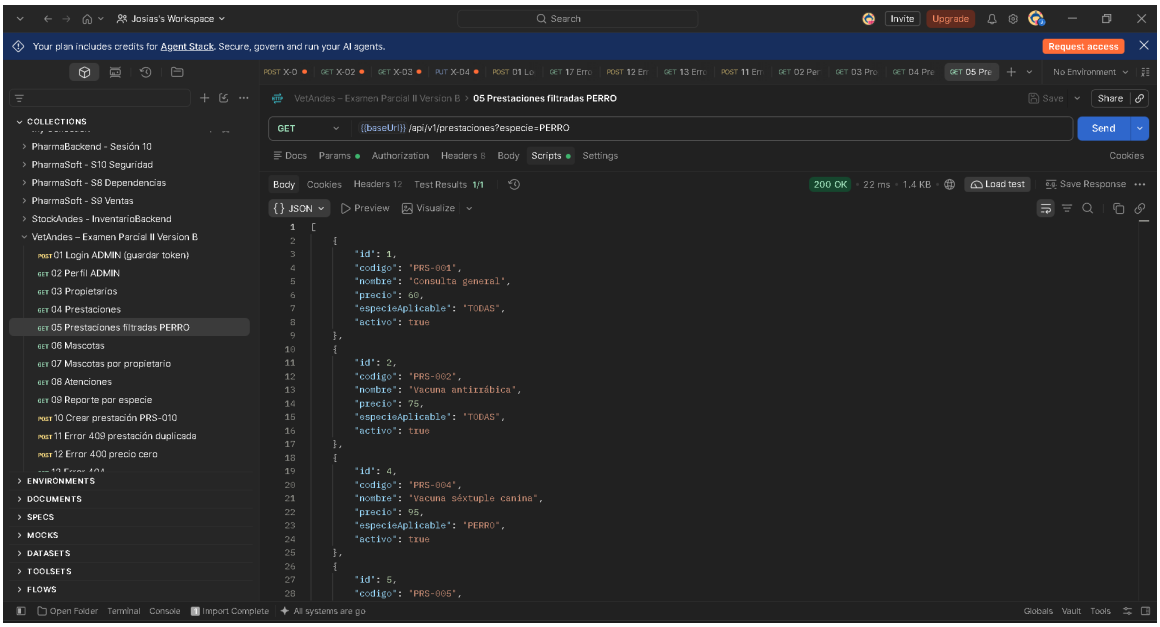
Se probo: GET /api/v1/propietarios como ADMIN.
Se observo: La API devolvió HTTP 200 y la lista de propietarios registrados.
Demuestra: La lectura de propietarios y la autenticación funcionan.

24. Postman - Lista de prestaciones, HTTP 200



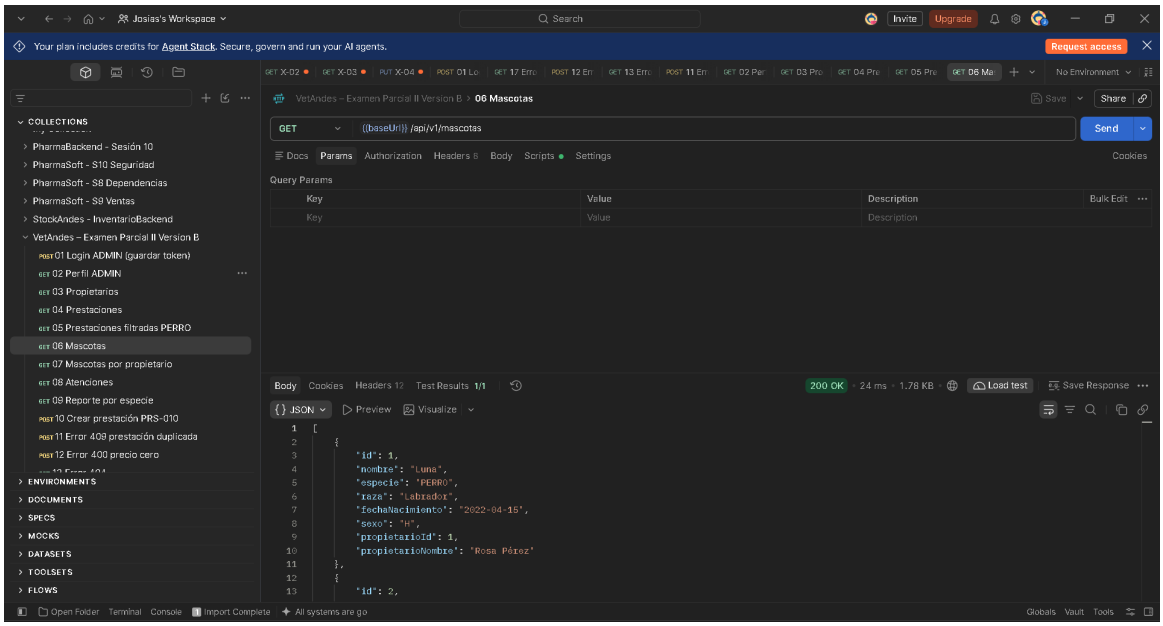
Se probo: GET /api/v1/prestaciones como ADMIN.
Se observo: HTTP 200, con códigos, nombres, precios y especies aplicables.
Demuestra: La consulta de prestaciones devuelve la información desde la API.

25. Postman - Prestaciones filtradas por PERRO, HTTP 200



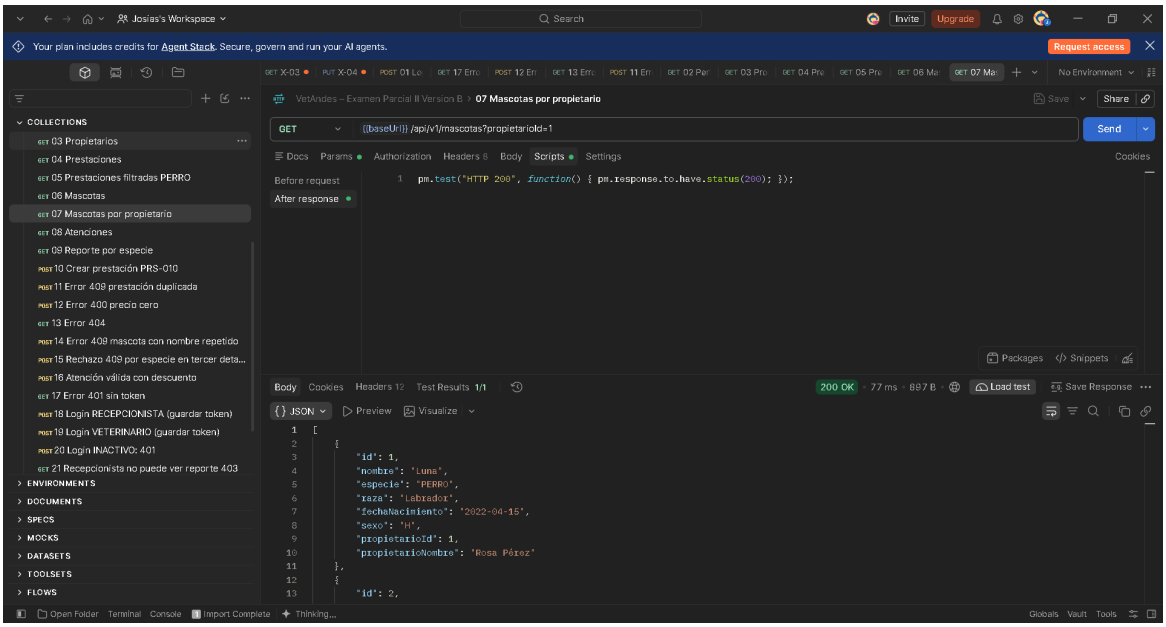
Se probo: GET /api/v1/prestaciones?especie=PERRO.
Se observo: HTTP 200; se incluyen prestaciones para PERRO y las de TODAS.
Demuestra: El filtro por especie devuelve prestaciones compatibles.

26. Postman - Consulta de mascotas, HTTP 200



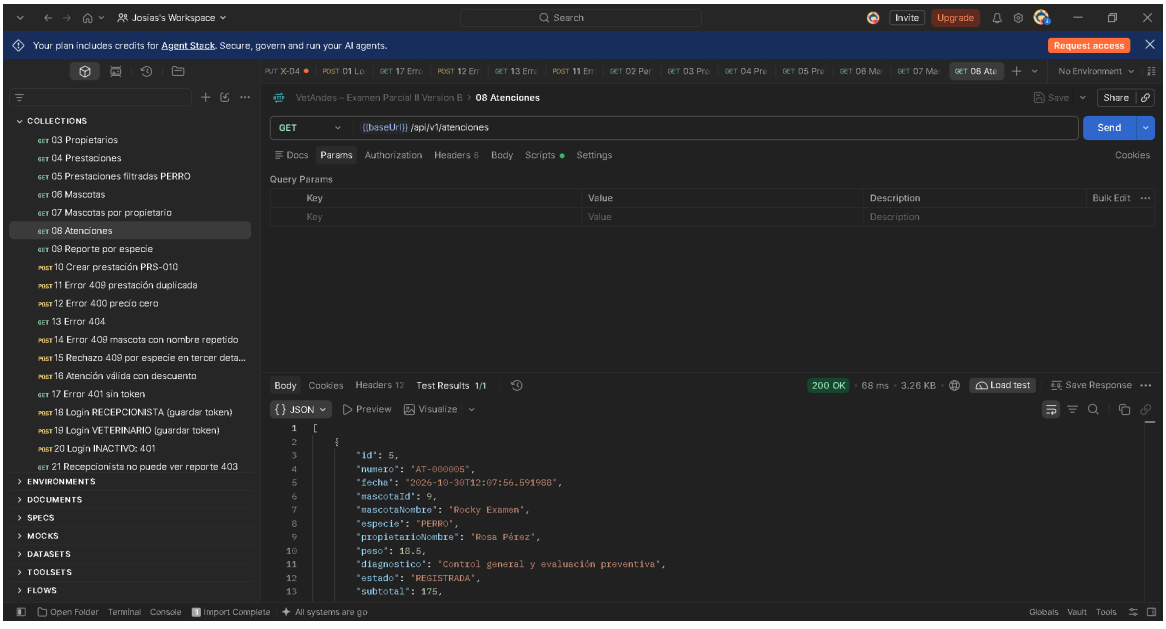
Se probo: GET /api/v1/mascotas como ADMIN.
Se observo: HTTP 200; lista con nombre, especie, raza y propietario.
Demuestra: La API de mascotas expone registros y su relación con propietarios.

27. Postman - Mascotas por propietario, HTTP 200



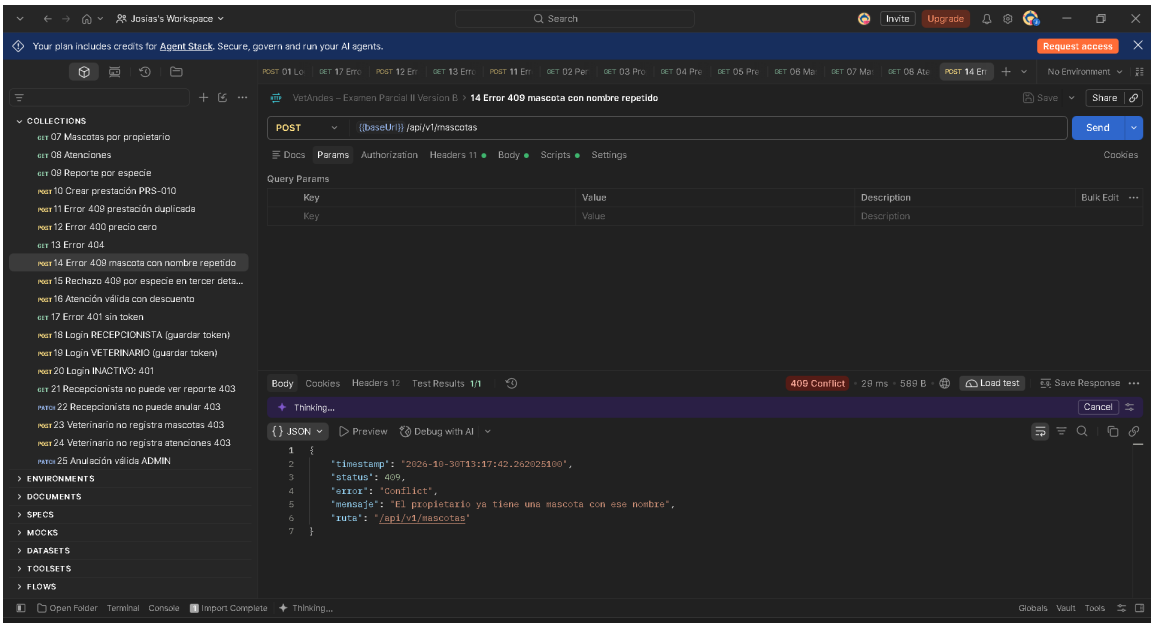
Se probo: GET /api/v1/mascotas?propietarioId=1.
Se observo: HTTP 200; lista de mascotas asociadas al propietario consultado.
Demuestra: El filtrado por propietario funciona en el backend.

28. Postman - Consulta de atenciones, HTTP 200



Se probo: GET /api/v1/atenciones con usuario autorizado.
Se observo: HTTP 200; aparece AT-000005, Rocky Examen y subtotal S/ 175.
Demuestra: La atención guardada se recupera desde la API.

29. Postman - Mascota duplicada, HTTP 409



Se probo: POST /api/v1/mascotas repitiendo el nombre para el propietario.
Se observo: HTTP 409 Conflict: ya existe una mascota con ese nombre.
Demuestra: El backend impide el duplicado y devuelve un error estructurado.